CEBU INSTITUTE OF TECHNOLOGY
UNIVERSITY

IT342-G6
System Integration and Architecture

System Design Document (SDD)

Project Title: JobConnect

Prepared By: Bigtasin, Van Andrae P.

Version: 1.0

Date: February 28, 2026
Status: Final

REVISION HISTORY TABLE

Version	Date	Author	Changes Made	Status
1.0	February 28, 2026	Van Andrae P. Bigtasin	Final review and corrections	Final

TABLE OF CONTENTS

Contents

EXECUTIVE SUMMARY 4
1.0 INTRODUCTION 5
2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION 5
3.0 NON-FUNCTIONAL REQUIREMENTS 8
4.0 SYSTEM ARCHITECTURE 9
5.0 API CONTRACT & COMMUNICATION 10
Authentication Endpoints 11
User Registration 11
User Login 11
6.0 DATABASE DESIGN 13
7.0 UI/UX DESIGN 14
8.0 PLAN 17

EXECUTIVE SUMMARY

1.1 Project Overview

JobConnect is an integrated job-matching and professional networking ecosystem. It bridges the gap between talent and opportunity by providing a synchronized experience across three tiers: a **Spring Boot** backend API, a **React** web dashboard for employers, and a **Kotlin/Android** mobile application for job seekers.

1.2 Objectives

1. Develop a secure, role-based platform for job seekers and employers.
2. Implement a **Three-Tier Architecture** to demonstrate robust system integration.
3. Provide real-time synchronization of job postings and application statuses via RESTful APIs.
4. Ensure data persistence using a structured relational database (MySQL/PostgreSQL).

1.3 Scope

Included Features:

- User authentication (JWT)
- Profile management (Resume/Bio)
- Job posting CRUD
- Search/Filter functionality
- Application tracking

Excluded Features:

- Real-time video calls
- AI-based resume parsing
- Payment gateway integration.

1.0 INTRODUCTION

1.1 Purpose

This document serves as the comprehensive design specification for the **JobConnect** system. It provides detailed requirements, architectural decisions, API contracts,

database design, and an implementation roadmap to guide development and ensure all components—backend, web, and mobile—integrate seamlessly.

2.0 FUNCTIONAL REQUIREMENTS SPECIFICATION

2.1 Project Overview

Project Name: JobConnect

Domain: Employment / Recruitment Tech

Primary Users: Job Seekers (Mobile) and Employers (Web)

Problem Statement: Job seekers need a streamlined way to apply for jobs on the go, while employers need a centralized system to manage applications without complex enterprise overhead.

Solution: An integrated three-tier platform that synchronizes data between an Android client and a React web management portal via a Spring Boot API.

2.2 Core User Journeys

Journey 1: Job Seeker (Mobile App)

1. User registers/logs in.
2. Users search for jobs by title or category.
3. The user views job details and clicks "Apply."
4. Users track the status of their application in their profile.

Journey 2: Employer (Web Dashboard)

1. Employer logs in.
2. The employer creates a new job post with requirements and salary.
3. The employer views a list of candidates who applied.
4. Employers update the candidate status (Shortlisted, Rejected, etc.).

2.3 Feature List (MoSCoW)

MUST HAVE

1. Role-based Authentication
2. Job Posting CRUD
3. Job Feed
4. Application Submission

SHOULD HAVE

1. Application Status Tracking
2. Profile Editing
3. Search Filters

COULD HAVE

1. Saved Jobs (Favorite list/Wishlist)
2. Basic Email Notifications

WON'T HAVE

1. In-app Real-time Chat
2. AI Resume Parsing
3. Video Interview Integration

2.4 Detailed Feature Specifications

Feature: User Authentication & Security (Mandatory Integration)

- Screens: Registration, Login, Profile Setup
- Fields: Email, Password, Full Name, Role Selection (Seeker/Employer), Google Login Button
- Integrations:

  * Google OAuth 2.0: Users can sign in via Google. The backend generates a custom JWT upon successful authentication.

- Security: Bcrypt password hashing (12 rounds) and JWT for stateless sessions.
- API Endpoints: POST /api/v1/auth/register, POST /api/v1/auth/login, POST /api/v1/auth/google, GET /api/v1/auth/me (Required)
- SMTP: Automatically sends a Welcome Email via SMTP upon successful account creation.

Feature: Job Discovery & External Insights (Mandatory Integration)

- Screens: Job Feed (Mobile), Trending Jobs (External)
- Integrations:

  * External API Consumption: Consumes a public API (e.g., GitHub Jobs or Adzuna) to display real-time "Global Trending Jobs" alongside local JobConnect listings.

- Search/Filter: Filter by Title, Category, and Employment Type.

- API Endpoints: GET /api/v1/jobs (Local), GET /api/v1/jobs/external (Third-party API data).

Feature: Candidate Profile & Document Management (Mandatory Integration)

- Screens: Profile View, Resume Upload
- Integrations:

  * File Upload: Seekers must upload a PDF Resume. The file is stored on the server file system and the path is linked to the database.

- Functions: Bio management, skill tagging, and resume download for employers.
- API Endpoints: GET /api/v1/profile, POST /api/v1/profile/resume (Multipart file upload).

Feature: Job Application & Notification Flow

- Screens: Job Detail, My Applications
- Process: Validates user role (Seeker only), checks for duplicate applications, and creates the link between User and Job.
- Integrations:

  * SMTP Notifications: Sends an Email Notification to the seeker immediately when an employer changes their application status to "Shortlisted."

- API Endpoints: POST /api/v1/applications, GET /api/v1/applications/user/{id}.

Feature: Employer Dashboard & Access Control

- Screens: Applicant List, Manage Listings
- RBAC (Role-Based Access Control):

  * API Level: Endpoints for creating jobs or viewing all applicants are restricted to "Employer" roles only.

  * UI Level: The "Post a Job" button is hidden for "Seeker" users.

- API Endpoints: POST /api/v1/jobs, PATCH /api/v1/applications/{id}/status.

2.5 Acceptance Criteria

AC-1: Successful User Registration (OAuth & Local)

- ➢ Given I am a new user
- ➢ When I enter valid details OR click "Sign in with Google"
- ➢ Then my account should be created in the database
- ➢ And I should receive a Welcome Email via SMTP.

AC-2: Mandatory File Upload (Resume)

- ➢ Given I am a Job Seeker on the Profile screen
- ➢ When I select a valid PDF file and click "Upload"
- ➢ Then the file should be stored on the server
- ➢ And the file path should be linked to my profile record.

AC-3: External API Integration

- ➢ Given I am on the Mobile Home Screen
- ➢ When the app loads the "Trending Jobs" section
- ➢ Then the system must successfully fetch and display data from the External GitHub Jobs API.

AC-4: Role-Based Access Control

- ➢ Given I am logged in as a "Job Seeker"
- ➢ When I try to access the "Post a Job" URL or API endpoint
- ➢ Then the system must return a 403 Forbidden error and hide the UI elements.

3.0 NON-FUNCTIONAL REQUIREMENTS

3.1 Performance Requirements

- API response time: ≤ 2 seconds for 95% of requests to ensure a smooth mobile experience
- Web page load time: ≤ 3 seconds on standard broadband connections.
- Mobile app cold start: ≤ 3 seconds on Android devices (API 24+).
- **Concurrency:** Support at least 100 concurrent users (Job Seekers and Employers).
- **Database Efficiency:** Queries must complete within 500ms to maintain real-time synchronization.

3.2 Security Requirements

- **Protocol:** HTTPS for all data transmission between mobile/web and the API.

- **Authentication:** Stateless JWT (JSON Web Token) authentication stored securely.
- **Encryption:** Password hashing using **Bcrypt** with a minimum of 12 salt rounds.
- **Injection Prevention:** Use of parameterized queries (JPA/Hibernate) to prevent SQL injection.
- **Authorization:** Role-Based Access Control (RBAC) to ensure Seekers cannot access Employer-only endpoints.

3.3 Compatibility Requirements

- **Web Browsers:** Latest two versions of Chrome, Firefox, Safari, and Edge.
- **Mobile (Android):** API Level 24+ (Android 7.0 Nougat) through latest.
- **Responsive Design:** Optimized for Mobile (360px+), Tablet (768px+), and Desktop (1024px+).

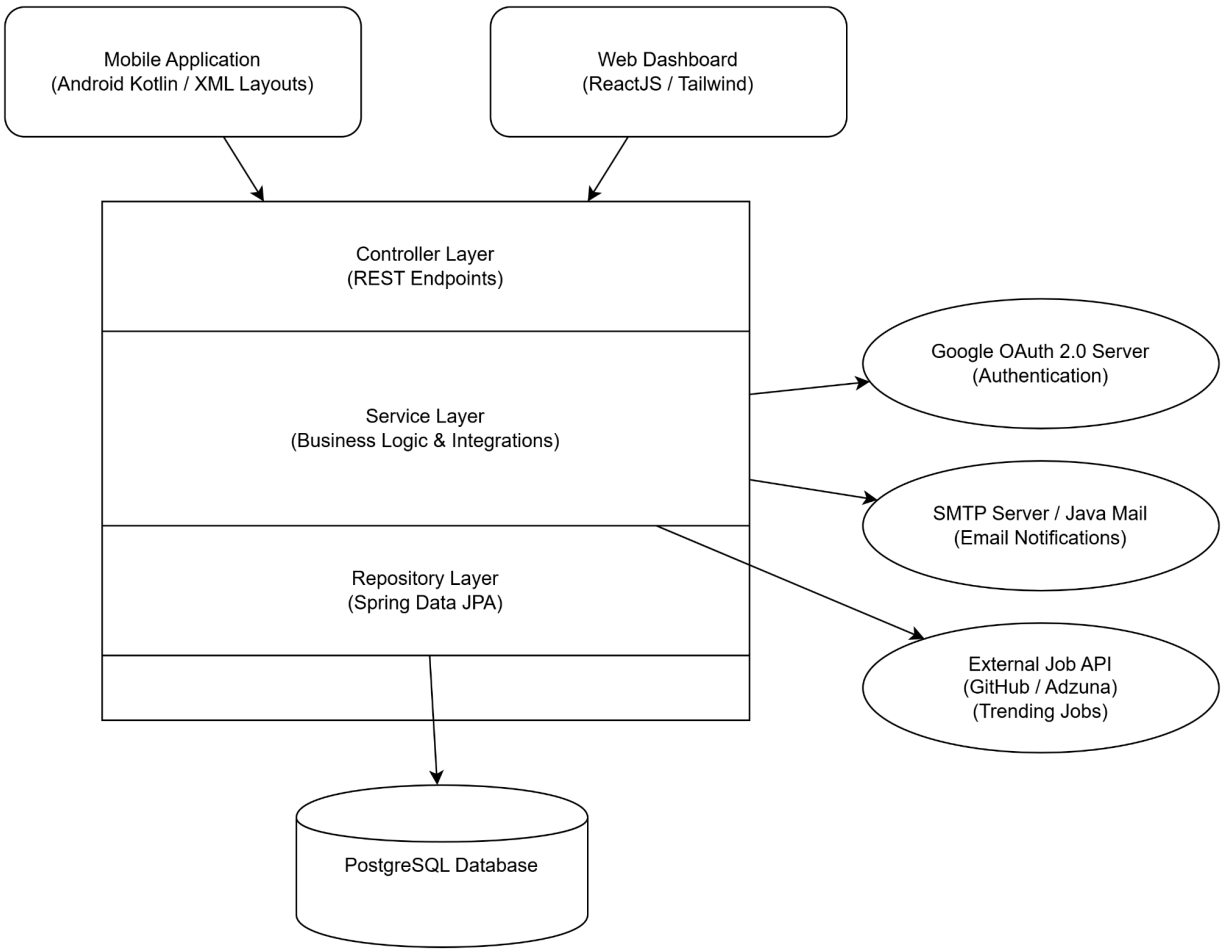
3.4 Usability Requirements

- **Efficiency:** A new user should be able to complete their profile and apply for their first job within 5 minutes.
- **Accessibility:** WCAG 2.1 Level AA compliance for the web management portal.
- **Mobile UX:** Minimum touch target size of 44x44px for all interactive buttons on the Android app.
- **Error Handling:** Descriptive error messages (e.g., "Email already in use") with clear recovery paths.

4.0 SYSTEM ARCHITECTURE

4.1 Component Diagram

Note: This should be a component diagram



Technology Stack:

- **Backend:** Java 17, Spring Boot 3.x, Spring Security (JWT), Spring Data JPA.
- **Database:** PostgreSQL 14+ (or MySQL).
- **Web Frontend:** React 18, TypeScript, Tailwind CSS, Axios.
- **Mobile (Strict XML Compliance):** Android Kotlin, **XML-Based UI Layouts** (API Level 34), Retrofit, ViewBinding/DataBinding.
- **Build Tools:** Maven (Backend), npm/yarn (Web), Gradle (Android).

JobConnect follows a **Layered Architecture pattern**. The **Controller Layer** manages the RESTful communication with the XML-based Android app and React web app. The **Service Layer** handles core logic and mandatory integrations (OAuth, SMTP, External API). The **Repository Layer** abstracts database interactions. **DTOs** are utilized across all layers to prevent the exposure of sensitive data like password hashes to the frontend.

5.0 API CONTRACT & COMMUNICATION

5.1 API Standards

Base URL	https://[server_hostname]:[port]/api/v1
Format	JSON for all requests/responses
Authentication	Bearer token (JWT) in Authorization header

Response Structure	{
	"success": boolean,
	"data": object|null,
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

5.2 Endpoint Specifications

Authentication Endpoints

User Registration

Description	User Registration
API URL	/auth/register
HTTP Request Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	"email": <email>,
	"password": <password>,
	"firstname": <firstname>,
	"lastname": <lastname>,
	}

Response Structure	{
	"success": boolean,
	"data": {
	user {
	"email": <email>,
	"firstname": <firstname>,
	"lastname": <lastname>,
	},

	“accessToken”: <token>,
	“refreshToken”: <token>,
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

User Login

Description	User Login
API URL	/auth/login
HTTP Method	POST
Format	JSON for all requests/responses
Authentication	None

Request Payload	{
	“email”: <email>,
	“password”: <password>,
	}

Response Structure	{
	"success": boolean,
	"data": {
	“user”: {
	“email”: <email>,
	“firstname”: <firstname>,
	“lastname”: <lastname>,
	“role”: <role>,
	},
	“accessToken”: <token>,
	“refreshToken”: <token>,
	},
	"error": {
	"code": string,
	"message": string,
	"details": object|null
	},
	"timestamp": string
	}

5.3 Error Handling

HTTP Status Codes

- 200 OK - Successful request
- 201 Created - Resource created
- 400 Bad Request - Invalid input
- 401 Unauthorized - Authentication required/failed
- 403 Forbidden - Insufficient permissions
- 404 Not Found - Resource doesn't exist
- 409 Conflict - Duplicate resource
- 500 Internal Server Error - Server error

Error Code Examples

Example 1	json
	{
	"success": false,
	"data": null,
	"error": {
	"code": "AUTH-001",
	"message": "Invalid credentials",
	"details": "Email or password is incorrect"
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Example 2	{
	"success": false,
	"data": null,
	"error": {
	"code": "VALID-001",
	"message": "Validation failed",
	"details": {
	"email": "Email is required",
	"password": "Must be at least 8 characters"
	}
	},
	"timestamp": "2024-01-28T10:30:00Z"
	}

Common Error Codes

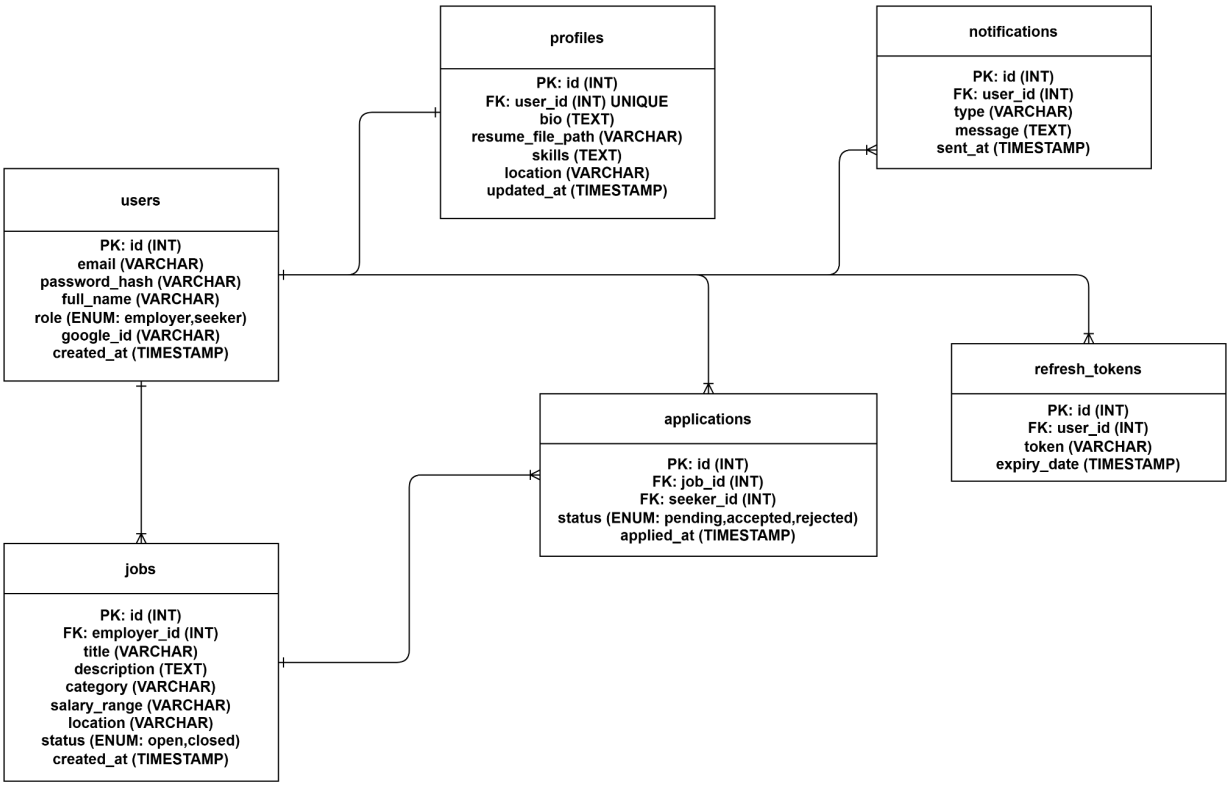
- AUTH-001: Invalid credentials
- AUTH-002: Token expired
- AUTH-003: Insufficient permissions

- VALID-001: Validation failed
- DB-001: Resource not found
- DB-002: Duplicate entry
- BUSINESS-001: Insufficient stock
- SYSTEM-001: Internal server error

6.0 DATABASE DESIGN

6.1 Entity Relationship Diagram

Note: This should be an ERD (Database Schema)



Detailed Relationships:

- **One-to-One:** User ↔ Profile (Professional bio and resume link).
- **One-to-Many:** Employer (User) → Jobs (Employer posts multiple jobs).
- **One-to-Many:** Job → Applications (One job receives many applicants).
- **One-to-Many:** Seeker (User) → Applications (One seeker applies to many jobs).

Key Tables:

1. **users:** id, email, password_hash (BCrypt), full_name, role, **google_id** (for OAuth integration), created_at

2. **profiles:** id, user_id (FK), bio, **resume_file_path** (Path to uploaded PDF on server), skills, location, updated_at
3. **jobs:** id, employer_id, title, description, salary, category, status
4. **applications:** id, job_id, seeker_id, status (PENDING/REVIEWED/REJECTED), applied_at

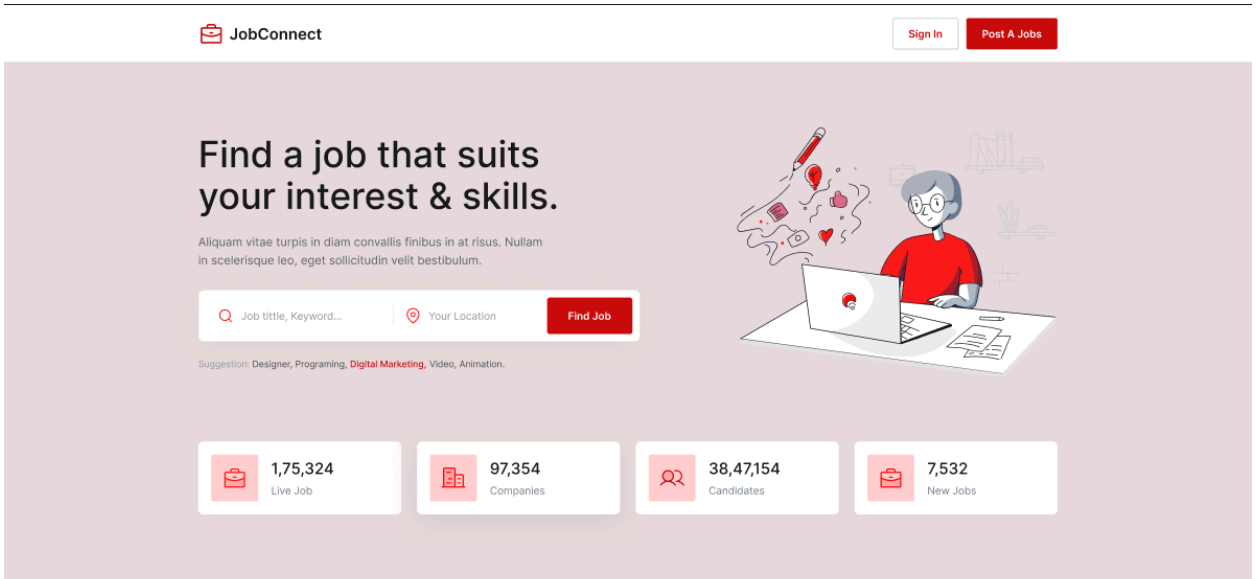
Table Structure Summary

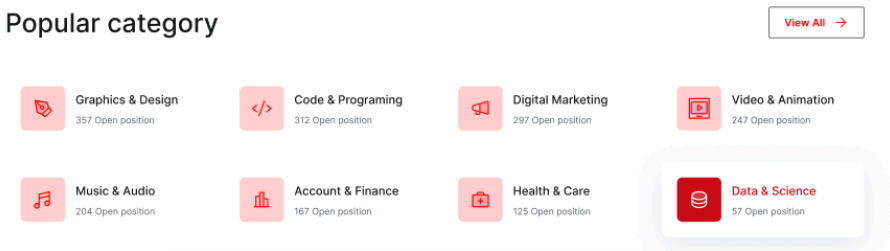
- **users:** id, email, password_hash (BCrypt), full_name, role (SEEKER/EMPLOYER/ADMIN), **google_id** (for OAuth integration), created_at
- **profiles:** id, user_id (FK), bio, **resume_file_path** (Linked to uploaded PDF on server), skills, location, updated_at
- **jobs:** id, employer_id (FK), title, description, category, salary_range, location, status (OPEN/CLOSED), created_at
- **applications:** id, job_id (FK), seeker_id (FK), status (PENDING/REVIEWED/SHORTLISTED/REJECTED), applied_at
- **notifications:** id, user_id (FK), type (EMAIL/SYSTEM), message, sent_at (Required for SMTP audit trail)
- **refresh_tokens:** id, user_id (FK), token, expiry_date

7.0 UI/UX DESIGN

7.1 Web Application Wireframes

Note: This should be wireframes from Figma





Employer Dashboard (Job Listings)

- **Header:** [Logo] [Search Applications] [Notifications] [User Profile]
- **Sidebar:** Dashboard, Post a Job, Manage Listings, Applicant Tracking, Settings
- **Content:** Job Management Table (Title, Status, Number of Applicants, Action Buttons: Edit/Delete/View)
- **Footer:** Links, Support, Copyright

Job Detail & Applicant View

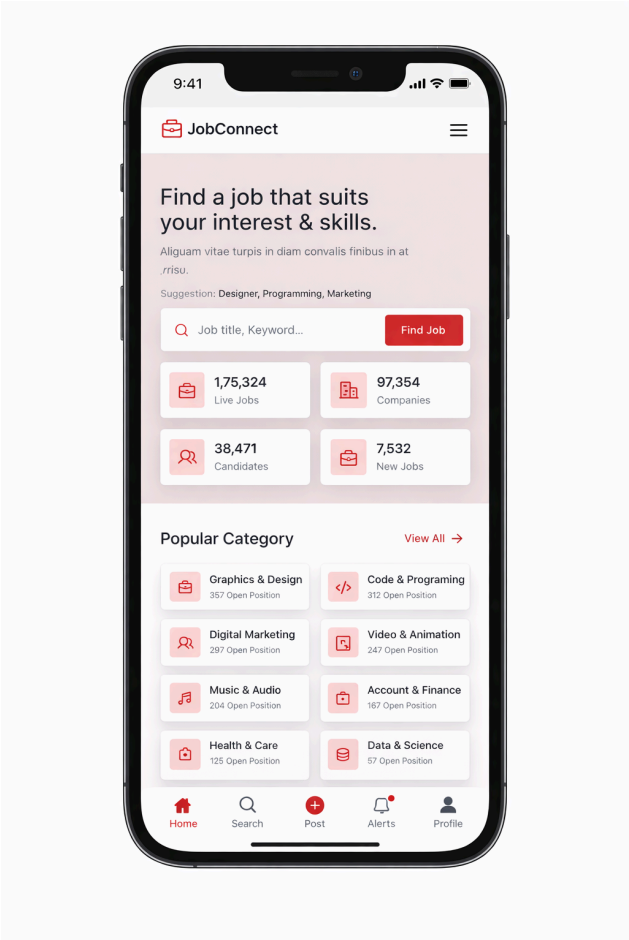
- **Back Button** to Dashboard
- **Job Summary:** Title, Category, Salary Range, and Description
- **Applicant List:** Seeker Name, Match Score (if applicable), Application Date, Status Toggle (Shortlist/Reject)
- **Action:** Button to view Seeker's Professional Profile/Resume

Post a Job Page

- **Form Fields:** Job Title, Department, Type (Full-time/Contract), Salary, Requirements, and Detailed Description
- **Action:** "Preview" and "Post Job" buttons

7.2 Mobile Application Wireframes

Note: This should be wireframes from Figma



Bottom Navigation

- [🏠 Home/Feed] [🔍 Search] [📄 My Applications] [👤 Profile]

Home Screen (Job Feed)

- **Search Bar:** Search by Title, Company, or Skill
- **Filters:** Location, Full-time/Remote, Salary range

- **Job Cards:** Company Logo, Job Title, Salary, "Easy Apply" button
- **Interaction:** Pull to refresh feed, Swipe to save for later

Job Detail Screen

- **Header:** Company info and Large "Apply" button
- **Content:** Expanded job description, requirements, and company culture
- **Sticky Footer:** "Apply Now" fixed button

My Applications Screen

- **Status List:** Cards showing applied jobs with progress indicators (Applied → Reviewed → Interviewing)
- **Actions:** Withdraw application, Follow-up button

Mobile-Specific Features:

- **Touch Optimization:** Large 44x44px buttons for "Apply"
- **Gesture Support:** Swipe cards to save/hide jobs
- **Design System:** Primary Blue (#2563EB), Secondary Purple (#7C3AED), Success Green (#10B981)

8.0 PLAN

8.1 Project Timeline

Phase 1: Planning & Design (Week 1-2)

- **Week 1:** Requirements (FRS/NFR) and System Architecture
- **Week 2:** API Specification, Database Schema (JobConnect ERD), and Figma Wireframes

Phase 2: Backend Development (Week 3-4)

- **Week 3:** Spring Boot setup, MySQL/PostgreSQL integration, and JWT Security
- **Week 4: Core Features:** User/Role endpoints, Job Posting CRUD, Application Submission logic

Phase 3: Web Application (Week 5-6)

- **Week 5:** React/Tailwind setup; Employer Login and Profile
- **Week 6: Employer Dashboard:** Job posting forms and Applicant management tables

Phase 4: Mobile Application (Week 7-8)

- **Week 7:** Android/Kotlin setup; **XML-Based UI Layouts** for Job Feed.
- **Week 8:** API integration (Retrofit); Search/Filter functionality and Profile editing

Phase 5: Integration & Deployment (Week 9-10)

- **Week 9:** Cross-platform testing (Mobile app applying $\rightarrow$ Web app receiving)
- **Week 10:** Cloud deployment (Heroku/Railway for API, Vercel for Web) and Project Submission

Milestones:

- **M1:** All design documents complete
- **M2:** Backend API fully functional and deployed
- **M3:** Web & Mobile apps integrated with the live database

Critical Path

The critical path identifies the must-complete tasks that directly impact the project deadline:

1. **Multi-Role Authentication System (Week 3):** Establishing JWT security for both Seekers and Employers.
2. **Job Management API (Week 3-4):** Developing the core CRUD operations for posting and retrieving job listings.
3. **Job Application Logic (Week 4):** Implementing the backend "bridge" that connects a Seeker ID to a Job ID.
4. **Cross-Platform Data Sync (Week 6):** Ensuring a job posted on the Web Dashboard is instantly visible on the Mobile Feed.
5. **Integration Testing (Week 9):** End-to-end testing of the seeker-to-employer communication flow.

Risk Mitigation

To ensure the project stays on track despite potential technical hurdles:

- **Feature Parity:** Start with the simplest working version of the "Apply" button before adding complex resume uploads.
- **Early Integration:** Test API endpoints with both the React and Android clients early to catch CORS or serialization errors.

- **Version Control:** Maintain a clean Git history with frequent pushes to avoid data loss during lab machine resets.
- **Core Focus:** Prioritize the Job Feed and Application submission over "Could Have" features like profile pictures or social sharing.
- **Environment Mocking:** Use tools like Postman to test the backend logic even if the Mobile/Web frontends are still under development.